Sesión 16: Patrón InheritedWidget (Mecanismo Nativo de Propagación de Información Descendente y Reactividad)

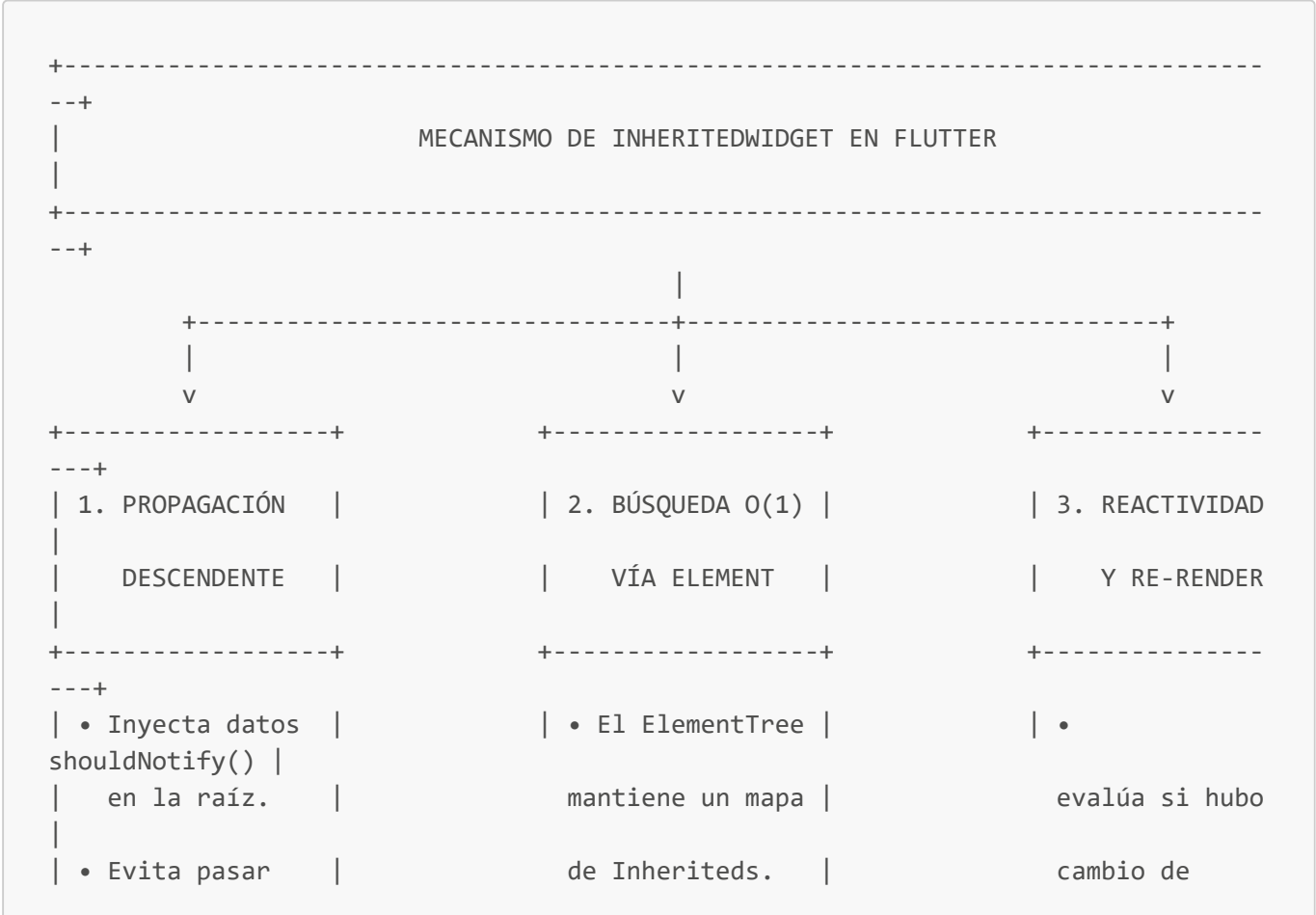
Introducción

En la sesión anterior analizamos las limitaciones de `setState()` y el problema del *Prop Drilling* al intentar compartir información en subárboles de widgets profundos. Flutter ofrece un mecanismo nativo de bajo nivel diseñado específicamente para resolver este problema sin recurrir a paquetes externos: la clase `InheritedWidget`.

En esta sesión estudiaremos el funcionamiento interno de `InheritedWidget`, cómo se integra con la jerarquía de `Element` en el motor de Flutter y el papel que desempeña el método `dependOnInheritedWidgetOfExactType`. Este patrón constituye el pilar sobre el cual se construyen los gestores de estado más populares del ecosistema (como `provider`) e interfaces clave del framework como `Theme.of(context)` y `MediaQuery.of(context)`.

Conceptos y Definiciones

El patrón `InheritedWidget` permite inyectar datos en la parte superior del árbol de widgets para que cualquier componente descendiente pueda leerlos u observarlos con una complejidad de búsqueda constante de $O(1)$.



```
datos. |
|   argumentos por |             | • Lectura rápida |             | • Notifica
solo  |
|   constructores. |               sin recorrer la  |               a los widgets
|
|                  |               jerarquía.       |               suscritos.
|
+------------------+             +------------------+             +--------------
---+
```

1. ¿Qué es un InheritedWidget?

- Es una clase inmutable abstracta especializada que extiende de `Widget`.
- Funciona como un contenedor de datos que se coloca en un punto superior del árbol de widgets.
- Permite que los widgets descendientes obtengan una referencia directa a sus propiedades mediante la consulta del `BuildContext`.

---

2. Búsqueda Eficiente $O(1)$ y el `ElementTree`

Cuando un widget llama a `Theme.of(context)` o consulta un `InheritedWidget` personalizado, Flutter no recorre iterativamente el árbol hacia arriba nodo por nodo.

- Cada `Element` en Flutter conserva un mapa de referencias internas a los `InheritedElement` ancestros más cercanos.
- Por lo tanto, la búsqueda del dato tiene una complejidad temporal de $O(1)$.
- Existen dos métodos principales de consulta expuestos por `BuildContext`:
- `dependOnInheritedWidgetOfExactType<T>()`: Registra al widget actual como dependiente. Cuando el `InheritedWidget` cambia, este widget se reconstruirá.
- `getInheritedWidgetOfExactType<T>()`: Obtiene el objeto de forma puntual **sin** suscribir al widget a futuros cambios de estado.

---

3. El Método `updateShouldNotify` y el Ciclo de Vida Reactivo

Para controlar la reactividad, todo `InheritedWidget` debe implementar el método `updateShouldNotify`:

```dart
@override
bool updateShouldNotify(covariant MiInheritedWidget oldWidget) {
  return propiedad != oldWidget.propiedad;
}
```

- **Evaluación de Cambios:** Compara las propiedades del widget viejo (`oldWidget`) contra las del widget nuevo.
- **Notificación Seleccionada:** Si el método retorna `true`, Flutter invoca automáticamente `didChangeDependencies()` y redibuja únicamente a los widgets descendientes que se hayan suscrito

explícitamente mediante `dependOnInheritedWidgetOfExactType`. Los widgets intermedios que no leen la información se omiten durante el ciclo de renderizado.

Ejemplo Completo con Código Fuente: Propagación de Usuario y Tema con InheritedWidget

El siguiente ejemplo demuestra la implementación de un `InheritedWidget` nativo que administra un modelo de sesión de usuario y un contador global. Combina un `StatefulWidget` (para mutar el estado) con un `InheritedWidget` (para propagarlo eficientemente sin *Prop Drilling*).

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const ContenedorSesionGlobal(
    child: DemoInheritedWidgetApp(),
  ));
}

/// Modelo de Datos para la Sesión
class PerfilUsuario {
  final String nombre;
  final String numeroBoleta;
  final String rol;

  const PerfilUsuario({
    required this.nombre,
    required this.numeroBoleta,
    required this.rol,
  });
}

/// 1. INHERITEDWIDGET: Hereda de la clase base nativa para propagar datos
class ContenedorSesionState extends InheritedWidget {
  final PerfilUsuario usuario;
  final int notificaciones;
  final VoidCallback onIncrementarNotificaciones;

  const ContenedorSesionState({
    super.key,
    required this.usuario,
    required this.notificaciones,
    required this.onIncrementarNotificaciones,
    required super.child,
  });

  /// Método estático de acceso que registra dependencia implícita O(1)
  static ContenedorSesionState of(BuildContext context) {
    final ContenedorSesionState? resultado =
        context.dependOnInheritedWidgetOfExactType<ContenedorSesionState>();
```

```
    assert(resultado != null, 'No se encontró ContenedorSesionState en el
contexto');
    return resultado!;
  }

  /// Evalúa si los widgets suscriptores deben reconstruirse tras una mutación
  @override
  bool updateShouldNotify(ContenedorSesionState oldWidget) {
    return notificaciones != oldWidget.notificaciones ||
        usuario != oldWidget.usuario;
  }
}

/// 2. STATEFULWIDGET ENVOLTORIO: Administra el estado mutable en el nivel
superior
class ContenedorSesionGlobal extends StatefulWidget {
  final Widget child;

  const ContenedorSesionGlobal({super.key, required this.child});

  @override
  State<ContenedorSesionGlobal> createState() => _ContenedorSesionGlobalState();
}

class _ContenedorSesionGlobalState extends State<ContenedorSesionGlobal> {
  int _contadorNotificaciones = 3;

  final PerfilUsuario _usuarioActual = const PerfilUsuario(
    nombre: 'Ozzy',
    numeroBoleta: '2026630001',
    rol: 'Profesor Titular ESCOM',
  );

  void _incrementarNotificaciones() {
    setState(() {
      _contadorNotificaciones++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return ContenedorSesionState(
      usuario: _usuarioActual,
      notificaciones: _contadorNotificaciones,
      onIncrementarNotificaciones: _incrementarNotificaciones,
      child: widget.child,
    );
  }
}

/// 3. APLICACIÓN RAÍZ
class DemoInheritedWidgetApp extends StatelessWidget {
  const DemoInheritedWidgetApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Patrón InheritedWidget',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaPerfilNivel1(),
    );
  }
}

/// WIDGET NIVEL 1: Pantalla Principal (Sin parámetros en constructor)
class PantallaPerfilNivel1 extends StatelessWidget {
  const PantallaPerfilNivel1({super.key});

  @override
  Widget build(BuildContext context) {
    // Lectura reactiva mediante el método .of()
    final estadoSesion = ContenedorSesionState.of(context);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Perfil Académico'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        actions: [
          Stack(
            alignment: Alignment.center,
            children: [
              IconButton(
                icon: const Icon(Icons.notifications),
                onPressed: estadoSesion.onIncrementarNotificaciones,
              ),
              Positioned(
                right: 8,
                top: 8,
                child: CircleAvatar(
                  radius: 10,
                  backgroundColor: Colors.red,
                  child: Text(
                    '${estadoSesion.notificaciones}',
                    style: const TextStyle(fontSize: 10, color: Colors.white),
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(width: 12),
        ],
```

```
      ),
      body: const TarjetaInformacionNivel2(),
    );
  }
}

/// WIDGET NIVEL 2: Intermedio totalmente desacoplado (No recibe propiedades)
class TarjetaInformacionNivel2 extends StatelessWidget {
  const TarjetaInformacionNivel2({super.key});

  @override
  Widget build(BuildContext context) {
    return const Padding(
      padding: EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Información del Usuario (Acceso O(1) nativo)',
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
          SizedBox(height: 8),
          Text(
            'Este widget intermedio "TarjetaInformacionNivel2" no contiene
parámetros '
            'en su constructor y no sufre de Prop Drilling.',
            style: TextStyle(color: Colors.grey, fontSize: 13),
          ),
          SizedBox(height: 20),
          // Subárbol profundo
          DetalleUsuarioNivel3(),
        ],
      ),
    );
  }
}

/// WIDGET NIVEL 3: Consumidor Profundo (Lee directamente del contexto)
class DetalleUsuarioNivel3 extends StatelessWidget {
  const DetalleUsuarioNivel3({super.key});

  @override
  Widget build(BuildContext context) {
    // Consulta directa al InheritedWidget sin pasar por el Nivel 2
    final sesion = ContenedorSesionState.of(context);
    final usuario = sesion.usuario;

    return Card(
      elevation: 2,
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: Theme.of(context).colorScheme.primaryContainer,
          child: const Icon(Icons.person),
        ),
```

```
          title: Text(
            usuario.nombre,
            style: const TextStyle(fontWeight: FontWeight.bold),
          ),
          subtitle: Text('Boleta: ${usuario.numeroBoleta}\n${usuario.rol}'),
          isThreeLine: true,
          trailing: IconButton(
            icon: const Icon(Icons.add_alert),
            onPressed: sesion.onIncrementarNotificaciones,
          ),
        ),
      );
    }
  }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de un Selector de Tema Global Nativo

1. Crea un `InheritedWidget` llamado `ThemeContainerState` que transporte un enumerado de tema (`ThemeMode`).
2. Diseña un `StatefulWidget` envoltorio `ThemeContainer` que contenga la variable de estado `_themeMode` y un método para conmutar entre `ThemeMode.light` y `ThemeMode.dark`.
3. Conecta el `ThemeMode` en la raíz de `MaterialApp` consultando el `InheritedWidget` y añade un botón en una pantalla secundaria profunda para alternating el tema global de la app.

Práctica 2: Comparativa de Rendimiento con `getInheritedWidgetOfExactType`

1. En el código del laboratorio, modifica el método `.of()` para ofrecer dos variantes:

- `ContenedorSesionState.of(context)` (usa `dependOnInheritedWidgetOfExactType`).
- `ContenedorSesionState.read(context)` (usa `getInheritedWidgetOfExactType`).

2. Utiliza `ContenedorSesionState.read(context)` en un widget que únicamente necesita invocar la función `onIncrementarNotificaciones` sin mostrar el valor del contador.
3. Inserta impresiones en consola (`debugPrint`) para verificar que el widget que usa `read()` **no** se re-renderiza cuando el contador incrementa, optimizando el ciclo de dibujado.

Resumen y Puntos Clave

- **Propagación Eficiente:** `InheritedWidget` es la solución nativa de Flutter para compartir información descendentemente sin pasar propiedades a través de constructores intermedios.
- **Acceso $O(1)$:** El `ElementTree` indexa las referencias a los `InheritedElement`, permitiendo búsquedas inmediatas a través de `BuildContext`.
- **Subordinación Reactiva:** El método `updateShouldNotify` determina si los widgets suscriptores deben reconstruirse tras una mutación.

- **`dependOn` vs `get`:** `dependOnInheritedWidgetOfExactType` crea una suscripción reactiva; `getInheritedWidgetOfExactType` únicamente realiza una lectura de solo una vez.

Bibliografía y Referencias

1. **Google LLC.** (2024). *InheritedWidget class - Flutter API Docs*. Recuperado de: https://api.flutter.dev/flutter/widgets/InheritedWidget-class.html
2. **Google LLC.** (2024). *BuildContext class - dependOnInheritedWidgetOfExactType*. Recuperado de: https://api.flutter.dev/flutter/widgets/BuildContext/dependOnInheritedWidgetOfExactType.html
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.